ДИСЦИПЛИНА «DEVOPS-ИНСТРУМЕНТЫ»

Лабораторная работа 1
Часть 1

API сервиса задач

Неделя 1. Контрольная точка 1. Подготовка и разделы 1-4 практической работы.

Сведения	Значение
Выполнила	Есімханова Аружан
Группа	25-ИСТК-2о
Дата отчёта	22.10.2026
Проект	DemoApi - сервис проектов и задач
Режим хранения	InMemory

ГРАНИЦЫ ПОДТВЕРЖДЕНИЯ

В отчёте разделены фактические HTTP-ответы и результаты CLI-проверок, анализ исходного кода и действия, требующие ручной проверки в Postman / Visual Studio. Технические результаты подтверждены сохранёнными данными; полный комплект обязательных UI-доказательств ещё не собран.

Публичный репозиторий проекта

Дата и время проверок приводятся в часовом поясе UTC+05:00. Даты сроков задач и граница просрочки определяются по UTC.

1. Цель, среда и структура проекта

Цель: запустить учебный backend DemoApi, обратиться к REST API, разобрать успешные ответы и ошибки, проверить срок жизни данных в памяти процесса.

Параметр	Условие методички / фактическая проверка
Платформа	ASP.NET Core / .NET 10, Windows; сервер Kestrel
Хранилище	InMemory; PostgreSQL и Docker в этой работе не требуются
Перенос учебного порта	В методичке 8080; в этой среде он занят Oracle. Для DemoApi выбран 8088, профиль http
Адрес проверенного API	http://localhost:8088; отдельный локальный проверочный запуск
HTTP-клиенты	Скрипт PowerShell и Postman CLI; браузер / Swagger предусмотрены методичкой
Интерфейсы Visual Studio / Postman	Обязательные скриншоты не получены; факт работы в этих окнах не заявляется

Архитектура учебного сервиса

Слой	Назначение
Controllers	Маршруты, проверка результата операции и HTTP-коды
Services	Прикладные операции и журналирование
Domain	Project, TaskItem, TaskItemStatus и правила переходов
Data	Интерфейсы репозиториев и реализация PostgreSQL
Data/InMemory	Общие списки в памяти процесса и реализации тех же интерфейсов

Решение открывается файлом DemoApi.slnx. В Program.cs по настройке Storage регистрируется подходящая реализация хранения. Благодаря общим интерфейсам HTTP-слой не зависит от способа сохранения данных.

2. Коллекция и успешные запросы

Создана импортируемая коллекция **demo-api** формата Postman v2.1. В текущей версии baseUrl по умолчанию равен http://localhost:8088. Учебный порт 8080 занят Oracle, поэтому профиль запуска, коллекция и примеры запросов согласованно используют 8088. projectId и taskId заполняются автоматически из ответов, имена проектов генерируются уникально.

В коллекции 24 запроса: 14 в базовом сценарии, 8 в сценарии статистики и 2 в отдельной дополнительной папке. В обязательном CLI-запуске выполнены первые две папки - 22 запроса. Импорт и вид коллекции в графическом Postman требуют отдельного скриншота.

Запрос	Ожид.	Факт	Результат
GET /api/projects	200	200	Пустой массив []
POST /api/projects	201	201	projectId=1; Location сохранён
POST /api/tasks	201	201	taskId=1; status=Todo
GET /api/tasks?projectId=1	200	200	Одна задача проекта 1
PATCH /api/tasks/1/status	200	200	status=InProgress
GET /api/projects/1	200	200	id=1; имя совпадает

Создание ресурса: 201 и Location

```
POST /api/projects
Content-Type: application/json
{"name": "lab01-20261021T194046965Z-all-7024bbae-part1"}
HTTP 201 Created
Location: http://localhost:8088/api/projects/1
```

Создан проект с id=1. Заголовок Location получен от сервера и указывает адрес нового ресурса. Аналогично POST задачи возвратил 201 и Location http://localhost:8088/api/tasks/1; начальный статус - Todo.

Таблица и фрагмент относятся к сохранённому HTTP-прогону от 22.10.2026 00:40:46. Они являются текстовым представлением фактических ответов, а не скриншотами Postman.

Демонстрация. Создание проекта в Swagger

Помимо коллекции выполнен отдельный запрос через Swagger: POST /api/projects с телом {"name":"Курс DevOps"}. Сервер возвратил 201, проект id=7 и Location http://localhost:8088/api/projects/7. Этот объект создан позднее и отличается от проекта id=1 в текстовом HTTP-прогоне.

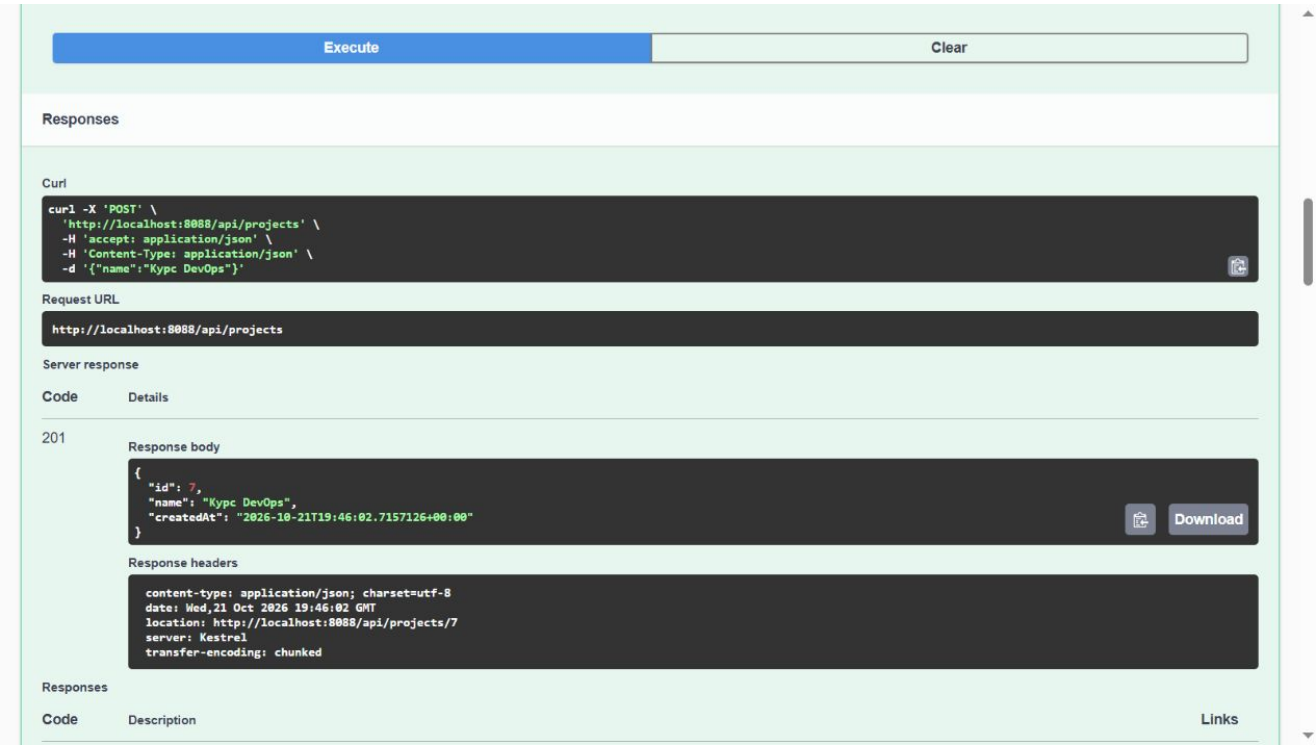


Рисунок 1. Swagger UI: настоящий результат Execute для создания проекта, HTTP 201, JSON и заголовки. Это отдельная демонстрация HTTP-клиента Swagger; скриншот коллекции Postman по-прежнему необходим.

Результат второго клиента

На снимке видны фактический Request URL, код Server response и заголовок location. Тем самым подтверждён запрос через Swagger из раздела «Другие клиенты того же API». Вызов из файла DemoApi.http через Visual Studio и состояние самой IDE отдельно не подтверждены.

3. Ошибки API: коды, errors и detail

Проверка	Код	Фактические данные ответа
POST /api/projects, пустое name	400	errors.Name: "The Name field is required."
Повторное имя проекта	409	Проект «lab01-20261021T194046965Z-all-7024bbae-part1» уже существует
Новая Todo-задача сразу в Done	409	Нельзя перевести задачу 2 из Todo в Done
GET /api/tasks/2147483647	404	title: "Not Found"; status: 404

Точные фрагменты JSON

```
{
"status": 400,
"errors": {
"Name": [
"The Name field is required."
]
}
}
```

Выше приведён фактический фрагмент ответа 400. В ответах также сохранены type, title и traceId; полные тела находятся в HTTP-доказательствах. Значение traceId не требуется для объяснения ошибки.

errors описывает ошибки полей модели запроса. **detail** объясняет конфликт состояния. Валидация name выполняется автоматически ASP.NET Core, а текст обоих ответов 409 задаёт контроллер. Для проверки отсутствующей задачи выбран int.MaxValue: на повторно используемом стенде он надёжнее фиксированного примера 999.

Почему нужна новая задача для проверки 409

Основная задача уже переведена из Todo в InProgress. Для запрещённого перехода создана отдельная задача id=2 со статусом Todo; PATCH с Done возвратил 409. Иначе переход InProgress в Done был бы допустим, и проверка не соответствовала бы условию.

Запрос к отсутствующей задаче возвратил 404. Это означает отсутствие ресурса, а не ошибку формата запроса и не внутренний сбой сервера.

4. Данные в памяти и перезапуск

До завершения процесса: HTTP 200, 4 проекта (22.10.2026 00:34:34). После нового запуска: HTTP 200, тело [] (22.10.2026 00:40:46). Это фактическая проверка перезапуска процесса; последовательность клавиш Shift+F5 / F5 в Visual Studio ею не подтверждается.

Этап	Сохранённое наблюдение
До остановки	GET /api/projects: 200; четыре ранее созданных проекта
После нового запуска	GET /api/projects: 200; []
Режим	InMemory; постоянное хранилище не подключено

Ответ: почему после перезапуска список пуст?

Проекты и задачи находятся в списках InMemoryStore в оперативной памяти. Хранилище зарегистрировано как singleton: оно общее для всех запросов одного работающего приложения, но не существует отдельно от процесса.

После завершения процесса его память освобождается. При новом запуске контейнер зависимостей создаёт новый InMemoryStore с пустыми списками и начальными счётчиками идентификаторов. Записи в файл или БД в этом режиме нет. Поэтому singleton обеспечивает общность данных между запросами, но не долговечность между запусками.

Порядок проверки по методичке

В Visual Studio нужно завершить отладку Shift+F5, снова запустить F5 и повторить GET /api/projects. В текущих доказательствах перезапуск подтверждается HTTP-снимками до и после отдельного процесса; использование указанных клавиш и состояние IDE не подтверждены.

ОГРАНИЧЕНИЕ РЕЗУЛЬТАТА

Вывод об отсутствии сохранения относится только к InMemory. Поведение PostgreSQL, томов Docker и Compose относится к следующим лабораторным и в этот отчёт не включено.

5. Проверки и состав сдачи

Проверка	Фактический итог
Скрипт HTTP, фаза All	23 запроса; passed=true; реальные ответы и Location сохранены
Postman CLI, обязательные папки	22 запроса, 67 проверок, 0 failed
Postman CLI, полная коллекция	24 запроса, 73 проверки, 0 failed; дополнительная папка включена
Исходный набор тестов	22 выполнено, 22 passed, 0 failed; InMemory
Повторяемость	Уникальные имена проектов; идентификаторы берутся из ответов API

Чек-лист контрольной точки 1

Требование	Статус и подтверждение
ФИО, группа, номер задания	Указаны в отчёте
Коллекция demo-api	JSON подготовлен; CLI-запуск подтверждён. Скриншот коллекции в Postman ещё нужен
Успешные запросы и Location	Подтверждены фактическими HTTP-ответами
400, 409, 409, 404; errors/detail	Подтверждены; точные данные приведены в разделе 3
Объяснение потери данных	Дано; HTTP-проверка перезапуска подтверждена
Swagger / Visual Studio	Создание проекта через Swagger подтверждено снимком; работа в IDE не подтверждена

Вывод. REST API выполняет проверенные операции создания, чтения и смены статуса, корректно возвращает ошибки валидации и конфликтов. Текстовые доказательства и коллекция готовы. Полная сдача контрольной точки требует настоящего скриншота коллекции Postman и предусмотренной преподавателем ручной демонстрации.

Приложение. Источники и воспроизведение

Материалы и версии

Основание: «Лаба 1,1.pdf» и «Лаба 1,3.pdf» - лекция; «Лаба 1,2.pdf» - практическая часть 1; «Лаба 1,4.pdf» - практическая часть 2. Условия на фотографиях уточняют состав контрольных точек. Вторая часть включает итоговый чек-лист первой.

Исходный код и документация
Обязательная версия: тег lab01-required-24, коммит f93b123

Архив исходного кода в репозитории

К отчётам прилагается отдельный архив **lab01-source.zip**: исходники src и tests, коллекция Postman, скрипт воспроизведения и сохранённые доказательства. Архив не содержит установленных зависимостей и результатов сборки.

Доказательство в архиве / репозитории	Содержание
evidence/lab01/20261021T194046965Z-all-7024bbae/ http-evidence.json	Запросы и ответы API; коды, заголовки Location, тела JSON и проверки
evidence/lab01/postman-required/console.txt	Локальный запуск Postman CLI: 22 запроса, 67 assertions, 0 failed
evidence/lab01/postman-final/console.txt	Полная коллекция Postman CLI: 24 запроса, 73 assertions, 0 failed
evidence/lab01/baseline/summary.json	Сводка исходных 22 тестов из TRX
evidence/lab01/required-24/summary.json	Сводка обязательных 24 тестов из TRX
evidence/lab01/restart/	Четыре проекта до перезапуска процесса; пустой список после

Сводки тестов очищены от служебных сведений о локальном пользователе и компьютере. Вывод Postman CLI подтверждает локальные запросы; публикация результатов в Postman Cloud не выполнялась.

Повторить проверку первой части

```
pwsh -File scripts/verify-lab01.ps1 -Phase Part1 `
-BaseUrl http://localhost:8088 -KeepData
```

Сервис должен быть запущен заранее. KeepData сохраняет созданные объекты для просмотра; без него скрипт удаляет только свои новые проекты. Скрипт не запускает и не останавливает сервер. Каждый запуск создаёт отдельный каталог доказательств.